

FIAP — Faculdade de Informática e Administração Paulista

SpaceHealth

Sistema Inteligente de Previsão de Risco de Dengue

Global Solution 2026.1

Integrantes:

- Giovanna Gomes Oliveira — RM567169
- Gabriel Coppola — RM568044
- Cloves Silva Filho — RM567250

Tutora: Sabrina Otoni

1. Introdução

A dengue é uma das doenças infecciosas de maior incidência no Brasil, com surtos diretamente relacionados a condições climáticas como temperatura elevada e precipitação. A identificação precoce de regiões com risco elevado permite que órgãos públicos e profissionais de saúde antecipem ações preventivas e reduzam o impacto da doença na população.

O **SpaceHealth** é uma solução desenvolvida para prever o risco de dengue em cidades brasileiras com base em dados climáticos. O sistema combina coleta de dados climáticos (via OpenWeather API e via um simulador de sensores ESP32), armazenamento relacional em SQLite, um modelo de Machine Learning treinado com **dados epidemiológicos e climáticos reais** e uma camada de visualização composta por uma interface web e um dashboard interativo.

Objetivo: construir um pipeline completo — da coleta de dados à previsão de risco — capaz de classificar o nível de risco de dengue (Alto, Médio ou Baixo) para uma determinada cidade a partir de variáveis climáticas.

2. Desenvolvimento

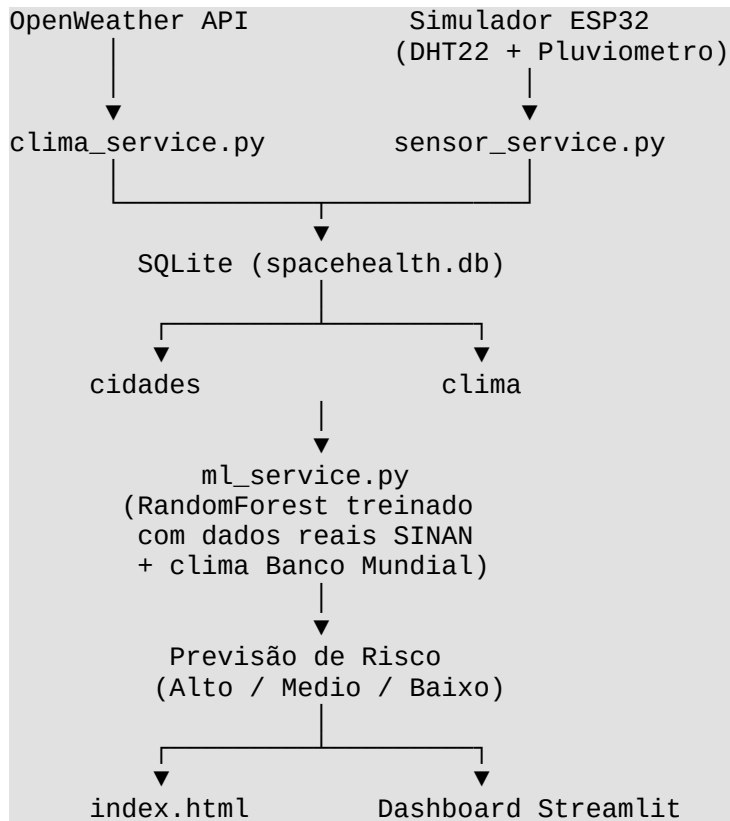
2.1 Arquitetura da Solução

O projeto foi organizado em uma arquitetura em camadas (padrão *repository / service*), separando o acesso a dados da lógica de negócio:

- **Repository** — acesso ao banco SQLite (conexão e operações CRUD).
- **Service** — regras de negócio: coleta climática, simulação de sensores e modelo de Machine

Learning.

- **Apresentação** — interface web estática (`index.html`) e dashboard interativo (Streamlit + Plotly).



2.2 Estrutura do Projeto

```
SpaceHealth/
├── src/
│   ├── setup_db.py           # cria banco e tabelas
│   ├── dashboard.py          # dashboard Streamlit + Plotly
│   ├── repository/
│   │   ├── conexao.py        # conexão SQLite
│   │   ├── cidade_repository.py # CRUD de cidades
│   │   └── clima_repository.py # leituras climáticas
│   ├── service/
│   │   ├── clima_service.py   # OpenWeather API
│   │   ├── sensor_service.py  # simulador ESP32
│   │   └── ml_service.py      # modelo de ML
│   └── templates/
│       └── index.html         # interface web
├── data/
│   ├── dataset_treino.csv     # base de treino (real)
│   └── chart.csv              # clima Banco Mundial
├── requirements.txt
├── spacehealth.db
└── docs/
```

2.3 Banco de Dados

Banco utilizado: **SQLite** — arquivo `spacehealth.db` gerado automaticamente, sem necessidade de servidor.

Tabelas:

Tabela	Colunas principais	Descrição
<code>idades</code>	<code>id</code> , <code>nome</code> , <code>estado</code>	Cadastro das cidades monitoradas
<code>clima</code>	<code>id</code> , <code>cidade_id</code> , <code>temperatura</code> , <code>chuva</code>	Leituras climáticas (API e sensores)

O arquivo `setup_db.py` cria as tabelas e popula as cidades iniciais:

```
from src.repository.conexao import conectar

SCHEMA = """
CREATE TABLE IF NOT EXISTS cidades (
    id      INTEGER PRIMARY KEY AUTOINCREMENT,
    nome    TEXT NOT NULL,
    estado  TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS clima (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    cidade_id   INTEGER NOT NULL REFERENCES cidades(id),
    temperatura REAL NOT NULL,
    chuva       REAL NOT NULL
);
"""

def criar_banco():
    conn = conectar()
    cursor = conn.cursor()
    cursor.executescript(SCHEMA)
    # ... seed das cidades
    conn.commit()
    conn.close()
```

Decisão técnica: a migração de PostgreSQL para SQLite eliminou a dependência de um servidor de banco, simplificando a execução do projeto — basta clonar o repositório e rodar, sem instalação ou configuração de credenciais.

2.4 Conexão com o Banco (`conexao.py`)

A camada de repositório centraliza a conexão com o SQLite. Todos os demais módulos a reutilizam:

```
import sqlite3

DB_PATH = "spacehealth.db"

def conectar():
    return sqlite3.connect(DB_PATH)
```

2.5 Cadastro de Cidades (cidade_repository.py)

Operação de inserção de cidades, com parâmetros vinculados (proteção contra SQL injection):

```
from src.repository.conexao import conectar

def inserir_cidade(nome, estado):
    conn = conectar()
    cursor = conn.cursor()
    cursor.execute(
        "INSERT INTO cidades (nome, estado) VALUES (?, ?)",
        (nome, estado)
    )
    conn.commit()
    cursor.close()
    conn.close()
```

2.6 Simulador ESP32 (sensor_service.py)

O módulo `sensor_service.py` simula um microcontrolador ESP32 equipado com dois sensores:

- **DHT22** — temperatura (°C)
- **Pluviômetro** — precipitação (mm)

O simulador gera leituras periódicas para todas as cidades cadastradas e as grava na tabela `clima`, replicando o fluxo que um ESP32 físico executaria via WiFi.

```
import random
import time
from src.repository.clima_repository import inserir_leitura

CIDADES = {1: "Sao Paulo", 2: "Campinas", 3: "Santos", 4: "Sorocaba"}
INTERVALO_SEGUNDOS = 5

def ler_temperatura():
    return round(random.uniform(22.0, 36.0), 1)

def ler_pluviometro():
    if random.random() < 0.70:
        return 0.0
    return round(random.uniform(1.0, 35.0), 1)

def executar():
    while True:
        for cidade_id, nome in CIDADES.items():
            temperatura = ler_temperatura()
            chuva = ler_pluviometro()
            inserir_leitura(cidade_id, temperatura, chuva)
            time.sleep(INTERVALO_SEGUNDOS)
```

Decisão técnica: o simulador usa 70% de probabilidade de chuva zero, refletindo que dias sem precipitação são mais frequentes — aproximando a distribuição gerada de dados climáticos reais.

2.7 Coleta de Dados Climáticos (clima_service.py)

O módulo `clima_service.py` consome a **OpenWeather API** para obter dados climáticos em tempo real de uma cidade:

```
import requests

API_KEY = "5f93e1156d7ed761ac74c3141c1e08b0"

def buscar_clima_atual(cidade="Sao Paulo"):
    url = (
        f"https://api.openweathermap.org/data/2.5/weather"
        f"?q={cidade}&appid={API_KEY}&units=metric&lang=pt_br"
    )
    resposta = requests.get(url)
    return resposta.status_code, resposta.json()
```

2.8 Dataset Real e Rotulagem do Risco

Diferente de uma base sintética, o modelo é treinado com **dados públicos reais**, cruzando duas fontes por mês:

- **Casos de dengue** — SINAN (Sistema de Informação de Agravos de Notificação), 2012–2021, com **12,4 milhões de notificações**. Os registros foram agregados por mês e filtrados pela confirmação (CLASSI_FIN).
- **Clima** — normais climáticas mensais do Brasil (Banco Mundial, 1991–2020): temperatura média e precipitação.

O rótulo `risco` é derivado do **tercil dos casos confirmados reais** (não de uma regra arbitrária): os 4 meses com menos casos são **Baixo**, os 4 intermediários **Medio** e os 4 com mais casos **Alto**.

Mês	Casos confirmados	Temperatura (°C)	Chuva (mm)	Risco
Jan	864.076	26,05	235,22	Medio
Fev	1.484.559	25,97	225,64	Alto
Mar	2.198.541	25,85	241,20	Alto
Abr	2.284.087	25,54	188,21	Alto
Mai	1.618.701	24,65	134,97	Alto
Jun	721.232	23,95	84,51	Medio
Jul	366.419	23,86	66,00	Medio
Ago	231.316	24,87	53,24	Baixo
Set	194.284	25,86	73,22	Baixo
Out	206.063	26,49	117,12	Baixo
Nov	241.166	26,23	159,69	Baixo
Dez	324.491	26,21	199,23	Medio

Achado relevante: o pico de chuva ocorre de janeiro a março, enquanto o pico de casos vai de fevereiro a maio — uma defasagem de aproximadamente um mês, compatível com o ciclo de proliferação do *Aedes aegypti* após as chuvas.

2.9 Modelo de Machine Learning (ml_service.py)

O modelo é um `RandomForestClassifier` treinado sobre o dataset real. As variáveis de entrada são **temperatura** e **chuva**.

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier

COLUNAS = ["temperatura", "chuva"]
DATASET_REAL = "data/dataset_treino.csv"

def dados_reais():
    df = pd.read_csv(DATASET_REAL)
    return df[["temperatura", "chuva", "risco"]]

def treinar_modelo(dados):
    X = dados[COLUNAS]
    y = dados["risco"]
    modelo = RandomForestClassifier()
    modelo.fit(X, y)
    return modelo

def prever_risco(modelo, temperatura, chuva):
    entrada = pd.DataFrame([[temperatura, chuva]], columns=COLUNAS)
    return modelo.predict(entrada)[0]
```

Decisão técnica — features: o modelo usa apenas variáveis com lastro em dados reais — temperatura e precipitação. São as duas grandezas presentes tanto nas normais climáticas do Banco Mundial quanto na coleta dos sensores, garantindo coerência entre treino e inferência.

Decisão técnica — algoritmo: o `RandomForest` foi escolhido por ser robusto a overfitting, não exigir normalização das features e produzir saída categórica direta (Alto/Medio/Baixo) sem necessidade de threshold manual.

Feature	Unidade	Relevância
Temperatura	°C	Temperaturas elevadas favorecem a reprodução do <i>Aedes aegypti</i>
Precipitação	mm	Chuva acumula água parada (principal criadouro)

Target: `risco` — classificação em três classes: Alto, Medio, Baixo.

2.10 Interface Web (index.html)

A interface `index.html` foi desenvolvida em HTML5, CSS3 e JavaScript puro. Apresenta o projeto, o pipeline da solução, as tecnologias e uma simulação de previsão por cidade, sem

necessidade de backend em execução.

Características visuais:

- Tema escuro (#0f172a como fundo)
 - Cards com bordas arredondadas e sombra
 - Paleta em azul céu (#38bdf8) para destaques
 - Layout responsivo com CSS Grid
-

2.11 Dashboard Interativo (Streamlit + Plotly)

O dashboard analítico foi construído com **Streamlit** e **Plotly**, funcionando como um "Power BI simplificado" em Python. Ele consome diretamente o banco SQLite e exibe:

- Previsão de risco por cidade em tempo real (a partir das últimas leituras dos sensores);
- KPIs (nº de leituras, cidades monitoradas, temperatura e precipitação médias);
- Gráficos de temperatura e precipitação médias por cidade, dispersão temperatura × precipitação e distribuição de precipitação;
- Tabela com os dados brutos.

O modelo de Machine Learning é treinado com o dataset real e aplicado às leituras dos sensores para classificar o risco de cada cidade no momento da consulta.

Para executar o dashboard:

```
streamlit run src/dashboard.py
```

3. Resultados Esperados

A execução do pipeline completo produz os seguintes resultados:

- O modelo classifica o risco de dengue de cada cidade a partir das leituras de temperatura e precipitação coletadas pelos sensores.
- O dashboard Streamlit exibe a distribuição de risco entre as cidades monitoradas, permitindo identificar regiões prioritárias para ação preventiva.
- A interface web permite uma consulta rápida e visual por cidade.

Impacto esperado: apoiar órgãos de saúde pública na priorização de recursos e ações de combate ao *Aedes aegypti* com base em dados climáticos objetivos e em padrões extraídos de casos reais, reduzindo o tempo de resposta a surtos.

4. Conclusão

O SpaceHealth demonstra que é possível construir um sistema de previsão epidemiológica funcional combinando ferramentas acessíveis: coleta de dados via API pública e simulação de sensores, banco de dados relacional SQLite, Machine Learning com Scikit-Learn e visualização em HTML e Streamlit.

O diferencial do projeto é o treinamento do modelo com **dados reais**: 12,4 milhões de notificações do SINAN cruzadas com as normais climáticas do Banco Mundial. O rótulo de risco não foi arbitrado — ele emerge dos casos efetivamente confirmados, o que confere fundamentação epidemiológica à solução.

Limitações identificadas:

- O dataset agregado é mensal e nacional; uma evolução natural seria regionalizar por estado ou município.
- O modelo usa duas variáveis climáticas (temperatura e precipitação); novas variáveis com lastro em dados reais poderiam ser incorporadas no futuro.

Próximos passos:

- Regionalizar o dataset (estado/município) para previsões mais granulares.
 - Automatizar a coleta climática com agendamento periódico.
 - Expor o modelo via API REST para integração com sistemas municipais de saúde.
-

5. Fontes de Dados

- Casos de dengue (SINAN, Brasil 2012–2021): <https://data.mendeley.com/datasets/2d3kr8zynf/4>
 - Dados climáticos históricos (Banco Mundial, Brasil 1991–2020): <https://climateknowledgeportal.worldbank.org/country/brazil/climate-data-historical>
-

6. Vídeo Demonstrativo

Link do vídeo no YouTube:

<https://youtu.be/v0I4eZE6fM8>

7. Repositório

Código-fonte disponível em:

<https://github.com/giovannag-tech/SpaceHealth->

Global Solution 2026.1 — FIAP Giovanna Gomes Oliveira (RM567169) | Gabriel Coppola (RM568044) | Cloves Silva Filho (RM567250)